PREPARATORIO ENADE 2026

TEORIA DOS GRAFOS

SQUAD 2

Davi Valdez Rodrigues · Gabriel Carvalho Bielawski · Gustavo Almeida de Moura · Larissa Pereira Ibiapino · Kauê Sanches Barbi

MATRIZ CURRICULAR

CAPÍTULO 1
Fundamentos e Propriedades
Vértices, arestas, funções, Handshake, planaridade e isomorfismo.

CAPÍTULO 2
Representação Computacional
Matriz e lista de adjacência.

CAPÍTULO 3
Algoritmos de Percurso
DFS, BFS e conectividade.

CAPÍTULO 4
Algoritmos de Otimização
Dijkstra, Bellman-Ford e Prim.

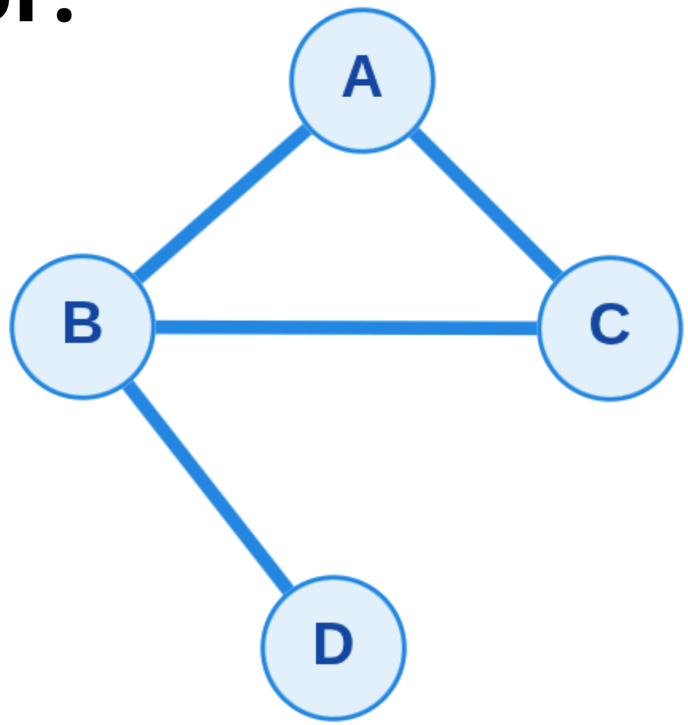
Capitulo 1

O que é um Grafo?

Grafo é uma estrutura matemática utilizada para representar **objetos e as relações entre eles.**

Um grafo é formado basicamente por:

- **Vértices (V):** representam os objetos ou entidades.
- **Arestas (E):** representam as conexões ou relações entre os vértices.
- Podemos representar um grafo por: **G=(V, E)**



- **Exemplo: V={A,B,C,D}**

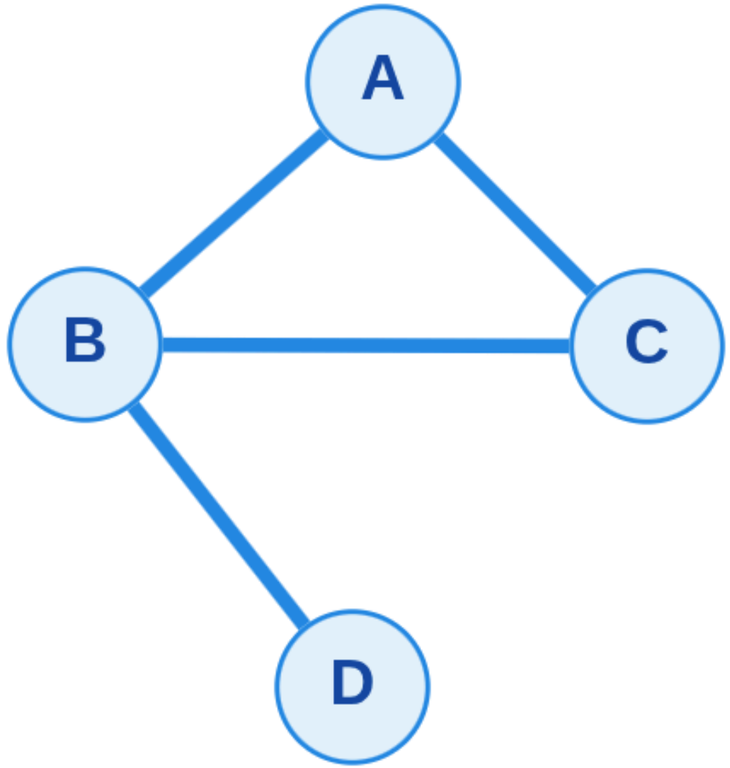
As arestas representam as conexões entre esses vértices.

Grau de um vértice

Grau de um vértice é a quantidade de arestas incidentes a esse vértice.
Aresta incidente: aresta que possui o vértice como uma de suas extremidades.

Graus dos vértices:

- A está conectado a B e C → grau 2
- B está conectado a A, C e D → grau 3
- C está conectado a A e B → grau 2
- D está conectado somente a B → grau 1



A soma dos graus desse grafo é: **2+3+2+1=8**

E o grafo possui 4 arestas: **2×4=8**

Handshake Lemma

A soma dos graus de todos os vértices de um grafo é igual ao **dobro do número de arestas.**

$$\sum_{v \in V} grau(v) = 2|E|$$

Por que isso acontece?
Cada aresta possui duas extremidades.
Portanto, cada aresta contribui **2** para a soma dos graus.

∑grau(v)=2+3+2+1=8
Como existem 4 arestas: **2 ∣ E ∣ =2(4)=8**

Se um grafo possui 10 arestas, então: **∑grau(v)=2(10)=20**
Não é necessário conhecer o grau de cada vértice para descobrir a soma.

Funções: Injetora, Sobrejetora e Bijetora

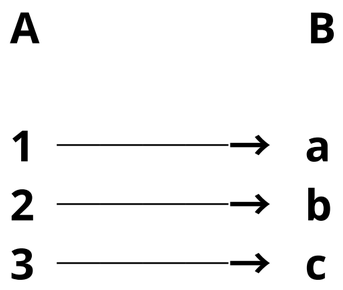
O que é uma função?

Imagine dois conjuntos: $A = \{1, 2, 3\}$ $B = \{a, b, c\}$

Uma função de A para B é representada por:

$$f : A \to B$$

Isso significa:
Para cada elemento de A, a função deve indicar exatamente um elemento de B.



Aqui temos:
f(1)=a
f(2)=b
f(3)=c

Regras Fundamentais

1. Todo elemento de A precisa ter uma seta.

2. Um elemento de A não pode apontar para dois elementos diferentes de B.

Por outro lado, vários elementos de A podem apontar para o mesmo elemento de B

1 → a
2 → b
3

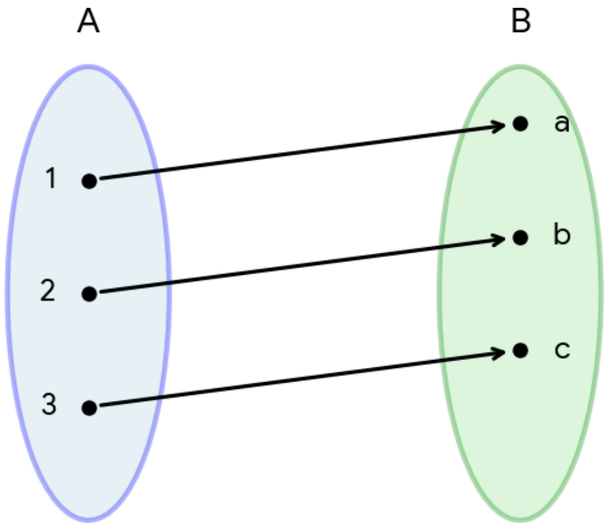
1 → a
1 → b
3 → c

1 ——→ a
2 ——→ a
3 ——→ b

Função Injetora

Uma função é **injetora** quando elementos diferentes do domínio possuem imagens diferentes.

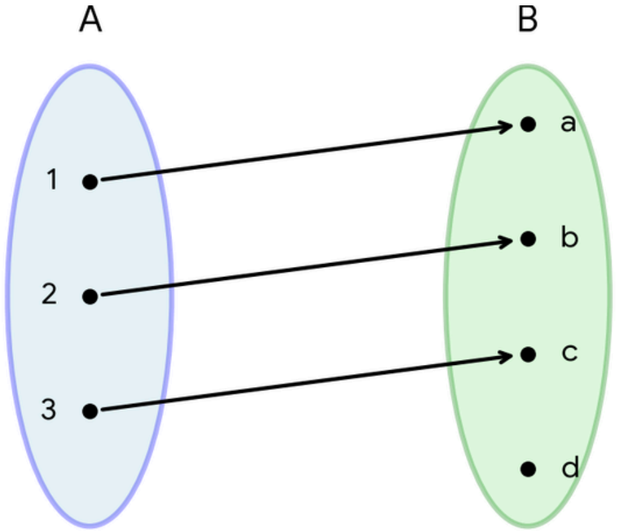
Em termos simples:
Não pode haver duas setas chegando no mesmo elemento de B.



Temos:

f(1)=a
f(2)=b
f(3)=c

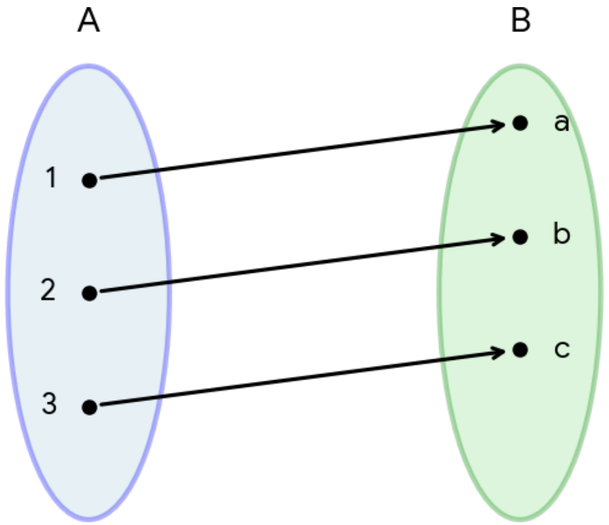
Nenhuma saída é repetida.
Portanto: **Injetora**



Uma função injetora não precisa utilizar todos os elementos de B.

Função Sobrejetora

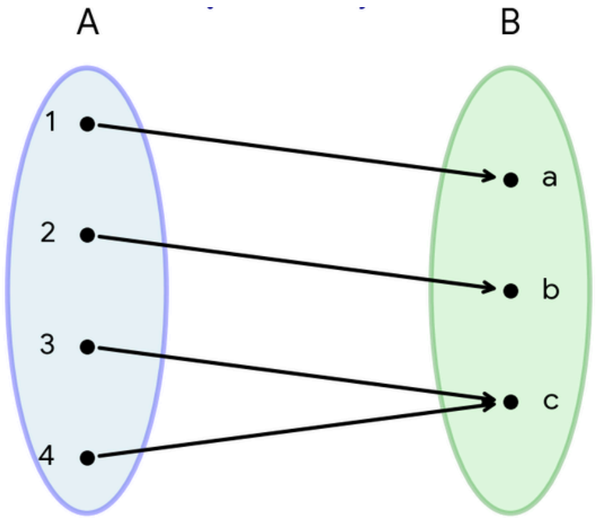
Todo elemento do contradomínio B é atingido por pelo menos um elemento de A.



Temos:

f(1)=a
f(2)=b
f(3)=c

Todos os elementos de B foram atingidos: **é sobrejetora.**



E se houver repetição? **Continua sendo sobrejetora**, porque a, b e c foram atingidos.

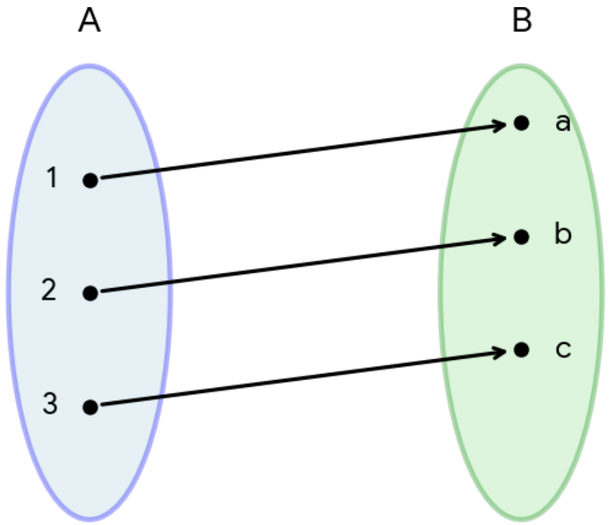
Função Bijetora

Uma função é bijetora quando é: **Injetora + Sobrejetora**

Ou seja:

Injetora
Nenhuma saída recebe duas entradas diferentes.

Sobrejetora
Nenhuma saída fica sem ser atingida.
Consequentemente, temos uma correspondência um para um entre os conjuntos.



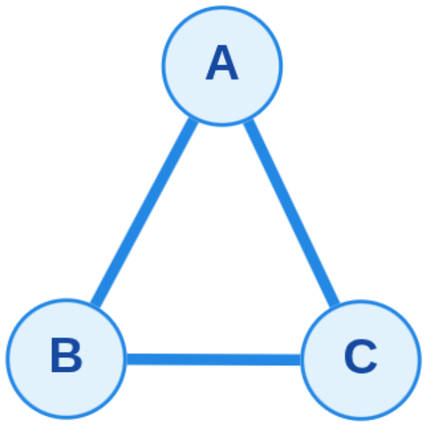
Cada elemento de A possui **uma única imagem** e cada elemento de B é **atingido exatamente uma vez**.

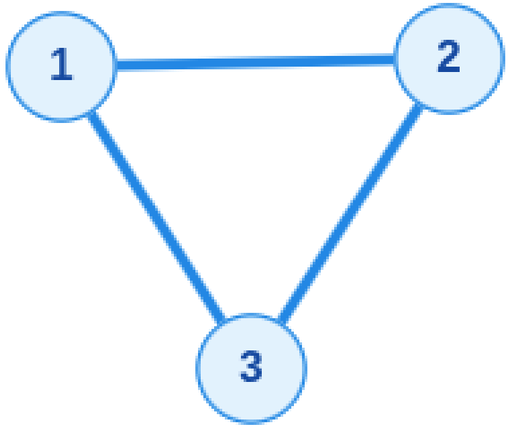
Injetora, Sobrejetora e Bijetora

Tipo de Função	No Conjunto A (Partida)	No Conjunto B (Chegada)
Injetora	Todos os elementos devem mandar **uma** flecha.	Os elementos recebem **no máximo uma** flecha (pode sobrar elemento livre).
Sobrejetora	Todos os elementos devem mandar **uma** flecha.	**Todos** os elementos devem receber **pelo menos uma** flecha (não pode sobrar ninguém).
Bijetora	Todos os elementos devem mandar **uma** flecha.	**Todos** os elementos recebem **exatamente uma** flecha (casamento perfeito).

Isomorfismo de Grafos

Dois grafos são isomorfos quando possuem a mesma estrutura de conexões, mesmo que sejam representados de formas diferentes.





Os dois possuem:

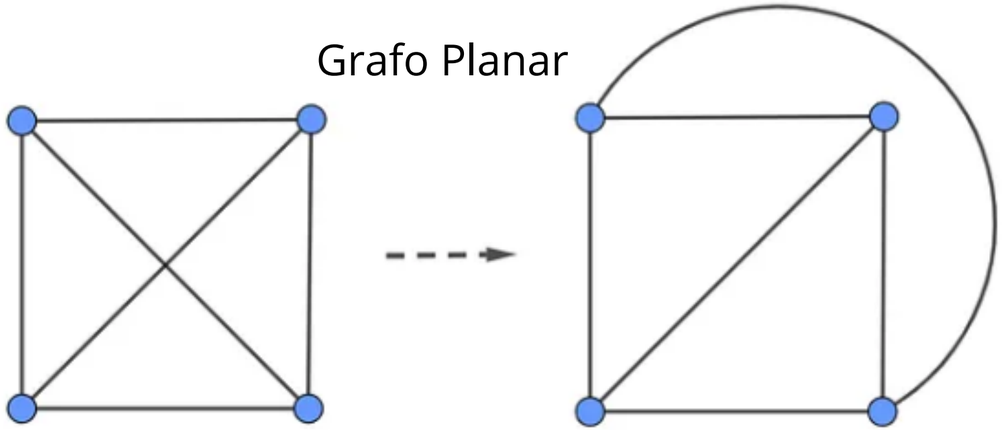
- 3 vértices;
- 3 arestas;
- os mesmos graus;
- as mesmas relações de adjacência.

Correspondência:
A↔1
B↔2
C↔3

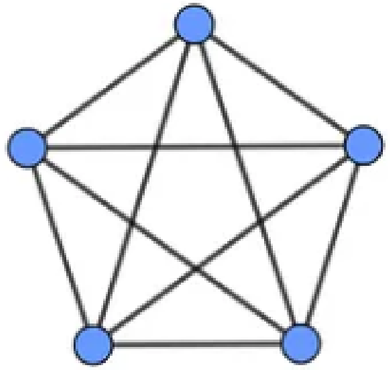
Planaridade dos Grafos

Um grafo é planar quando pode ser desenhado no plano de forma que nenhuma aresta cruze outra, exceto nos próprios vértices.

Grafo Planar



Grafo Não Planar

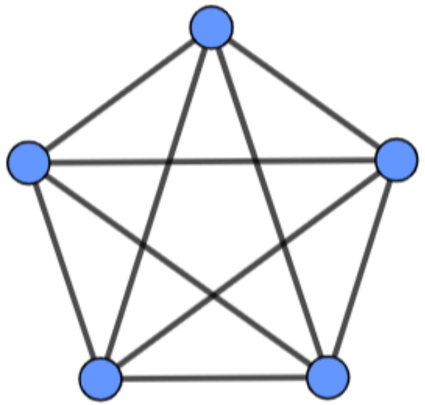


Importante:

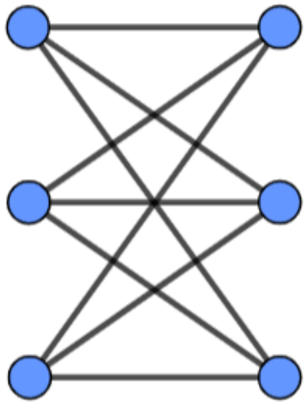
- O desenho pode ser reorganizado.
- As arestas podem ser curvas.
- O que importa é **manter as mesmas conexões** entre os vértices.
- Não basta um desenho específico ter cruzamentos: é preciso verificar se **existe alguma forma de desenhá-lo sem cruzamentos**.

Teorema de Kuratowski

O Teorema de Kuratowski permite caracterizar grafos não planares a partir de determinadas estruturas fundamentais.



K_5



$K_{3,3}$

“Um grafo possui uma **estrutura equivalente a uma subdivisão de K5 ou K3,3.** O que podemos concluir sobre sua planaridade?”

Resposta**: ele é não planar.**

QUESTÃO 1 - ENADE

SITUAÇÃO-PROBLEMA

Uma equipe de desenvolvimento está modelando uma rede de comunicação entre diferentes pontos de uma infraestrutura computacional. Para representar a rede, foi utilizado um grafo não direcionado G=(V,E), no qual os vértices representam os pontos da rede e as arestas representam as conexões entre eles.
Durante a análise da representação, a equipe identificou que o grafo possui 10 vértices e 15 arestas. Além disso, após reorganizar visualmente os vértices e as arestas, foi identificada uma subdivisão de K3,3 na estrutura do grafo.

COMANDO

Considerando os conceitos de grau dos vértices, Handshake Lemma e planaridade, analise a situação apresentada e assinale a alternativa que apresenta a conclusão correta.

ALTERNATIVAS

A) A soma dos graus dos vértices é 15, e o grafo pode ser planar, pois os cruzamentos das arestas podem ser eliminados pela reorganização dos vértices.
B) A soma dos graus dos vértices é 30, e o grafo é não planar devido à presença de uma subdivisão de K3,3.
C) A soma dos graus dos vértices é 25, e o grafo é não planar porque possui número ímpar de arestas.
D) A soma dos graus dos vértices é 30, porém não é possível determinar a planaridade do grafo apenas pela identificação de uma subdivisão de K3,3.

QUESTÃO 1 - GABARITO

A) Incorreta porque a soma dos graus não é igual ao número de arestas; cada aresta contribui com 2 para a soma dos graus.

B) A soma dos graus dos vértices é 30, e o grafo é não planar devido à presença de uma subdivisão de K3,3.

C) Incorreta porque a quantidade ímpar de arestas, por si só, não determina que um grafo seja não planar.

D) Incorreta porque a identificação de uma subdivisão de K3,3 é justamente uma condição suficiente para caracterizar o grafo como não planar.

QUESTÃO 1 - GABARITO

Gabarito: B

Justificativa:

Pelo Handshake Lemma:

$$\sum_{v \in V} grau(v) = 2|E|$$

Como o grafo possui 15 arestas:

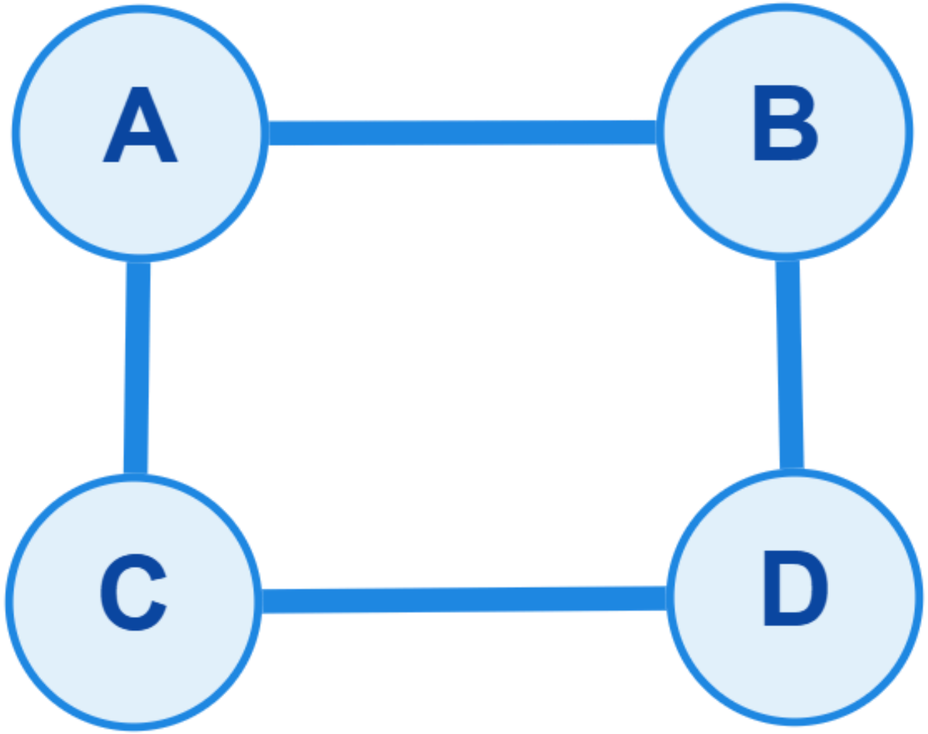
$$\sum grau(v) = 2(15) = 30$$

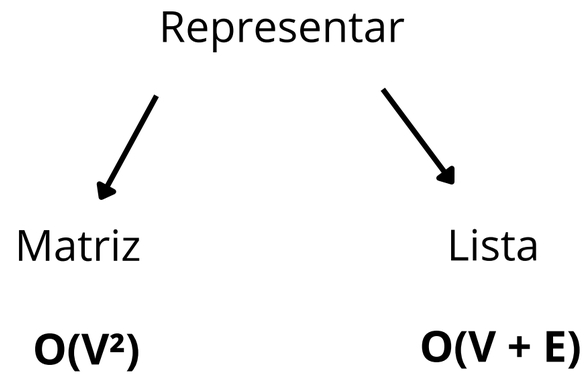
Além disso, a identificação de uma **subdivisão de K3,3** permite concluir, pelo Teorema de Kuratowski, que o grafo é **não planar.**

Capitulo 2

Representação Computacional

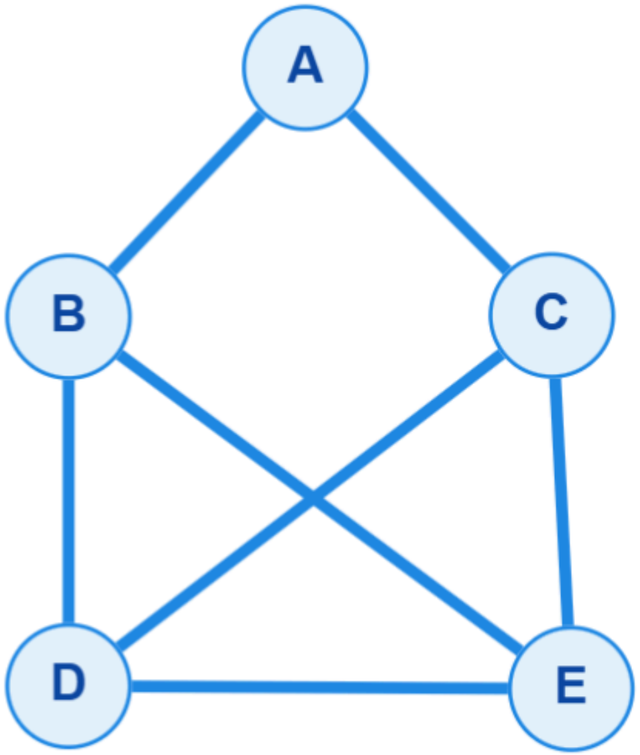
Como representar um grafo para o computador?





Espaço • Tempo • Eficiência

Matriz de Adjacência

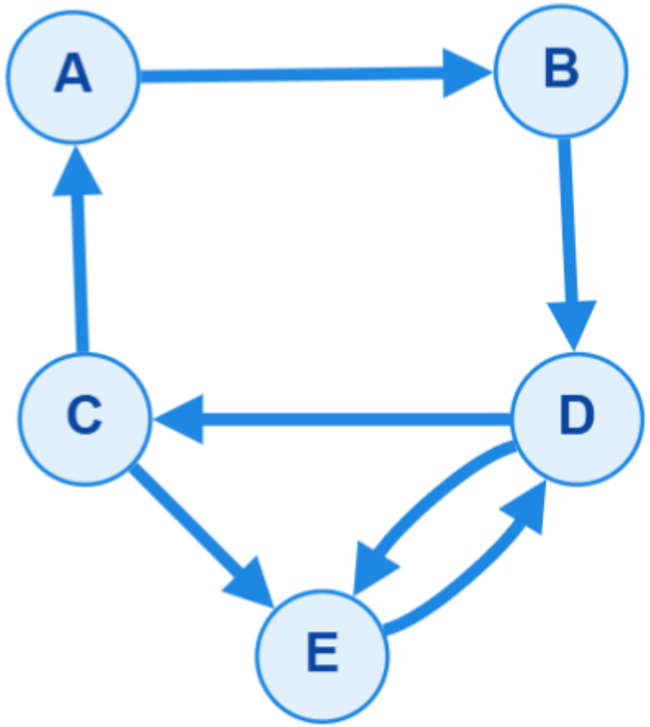


	A	B	C	D	E
A	0	1	1	0	0
B	1	0	0	1	1
C	1	0	0	1	1
D	0	1	1	0	1
E	0	1	1	1	0

1 → conexão

0 → sem conexão

Matriz de Adjacência (Grafos Direcionados)

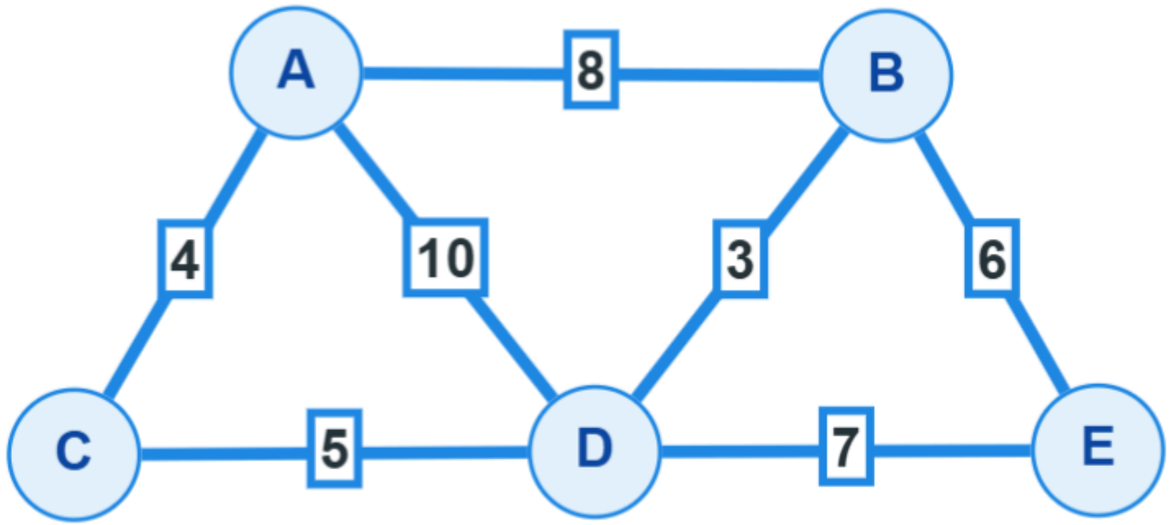


	A	B	C	D	E
A	0	1	0	0	0
B	0	0	0	1	0
C	1	0	0	0	1
D	0	0	1	0	1
E	0	0	0	1	0

1 → conexão

0 → sem conexão

Matriz de Adjacência (Grafos Ponderados)

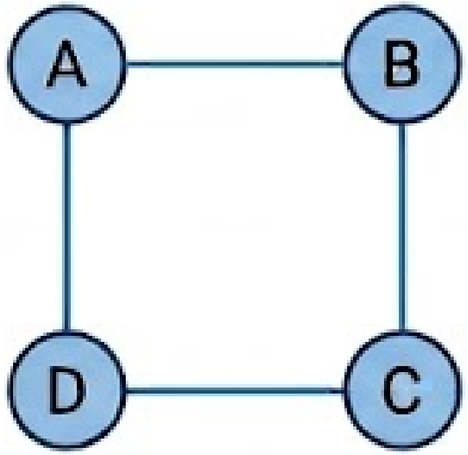


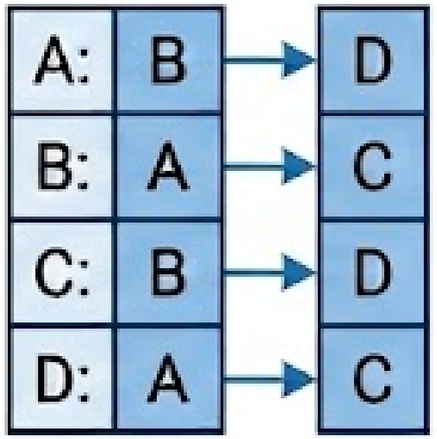
	A	B	C	D	E
A	0	8	4	10	0
B	8	0	0	3	6
C	4	0	0	5	0
D	10	3	5	0	7
E	0	6	0	7	0

valor → peso da aresta

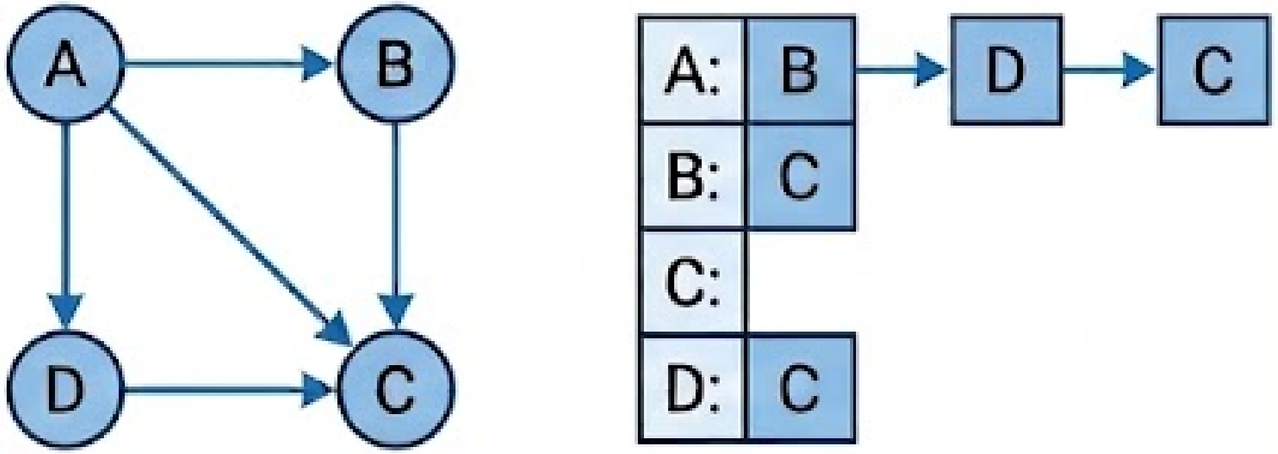
0 → sem conexão

Lista de Adjacência



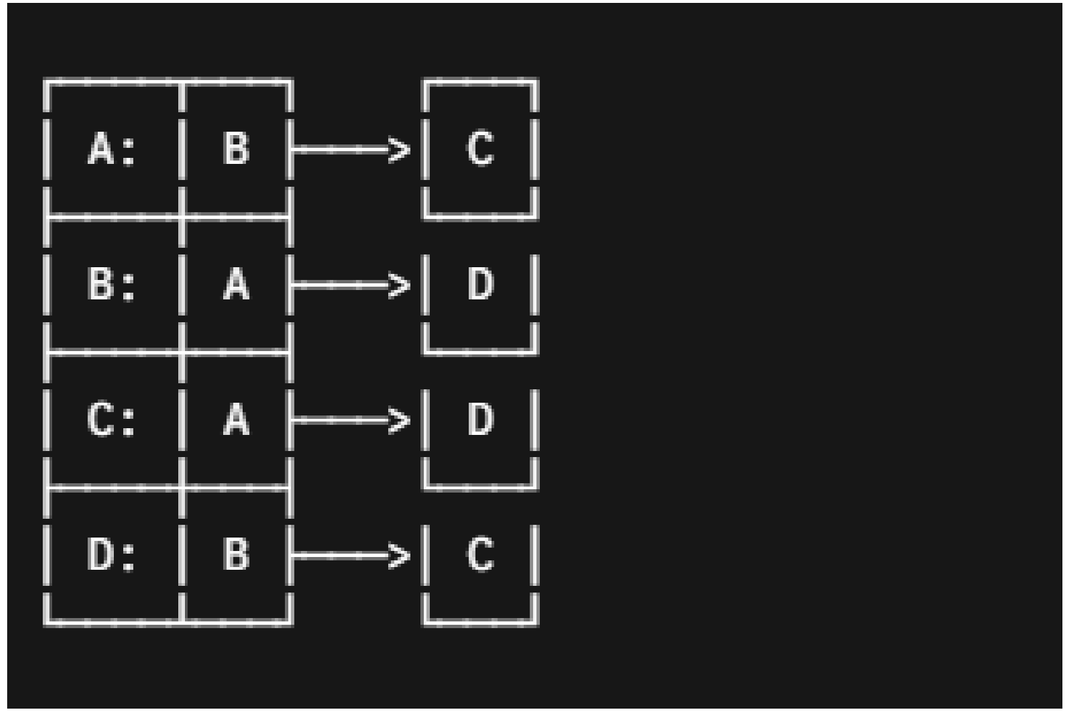


Lista de Adjacência (Grafos Direcionados)



Matriz X Lista

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0



- **Todas as possibilidades**
- **Acesso direto**
- **Pode gastar mais memória**

- **Só conexões existentes**
- **Evita posições desnecessárias**
- **Pode economizar memória**

Quando utilizar?

MUITAS CONEXÕES
↓
MATRIZ

POUCAS CONEXÕES
↓
LISTA

QUESTÃO 2 - ENADE

SITUAÇÃO-PROBLEMA

Uma equipe de engenharia de software está projetando o módulo de conexões e rotas de uma aplicação. O sistema precisa armazenar e manipular um grafo na memória, onde as interseções representam vértices e as rotas representam arestas direcionadas. Diante da preocupação com a alocação de memória e com a velocidade de acesso às conexões, a equipe analisa a adoção de Matrizes de Adjacência ou Listas de Adjacência.

COMANDO

Analise as afirmações a seguir sobre as estruturas de representação computacional de grafos e a tomada de decisão da equipe, identificando-as como Verdadeiras (V) ou Falsas (F):

( ) A Matriz de Adjacência aloca uma tabela com dimensão fixa proporcional a V x V (onde V é o número de vértices), reservando espaço na memória para todas as conexões possíveis, existam elas ou não.
( ) Em grafos direcionados, a Matriz de Adjacência é naturalmente simétrica, de modo que a posição (Linha A, Coluna B) terá sempre o mesmo valor que a posição (Linha B, Coluna A).
( ) A Lista de Adjacência é mais eficiente em termos de armazenamento quando o grafo é esparso (possui poucas conexões em relação ao número de vértices), pois guarda apenas os vértices vizinhos com os quais realmente existe conexão.
( ) A verificação direta da existência de uma conexão entre dois vértices específicos é operacionalmente mais rápida em uma Lista de Adjacência do que em uma Matriz de Adjacência, pois a lista permite acesso indexado direto em tempo constante O(1).

Assinale a alternativa com a sequência correta.

ALTERNATIVAS

A) V – F – V – F

B) V – V – V – F

C) F – F – V – V

D) V – F – F – V

QUESTÃO 2 - GABARITO

A) A matriz aloca tamanho V x V fixo; grafos direcionados geram matrizes assimétricas; a lista economiza memória em grafos esparsos ao guardar só o que existe; e o acesso direto O(1) é vantagem da matriz, não da lista.

B) Considera verdadeira a segunda afirmação, ignorando a assimetria criada pelas conexões direcionadas, que é o erro conceitual mais frequente do tópico. E aceita a falsa premissa de que o sentido da seta não altera a posição na matriz.

C) Nega a primeira afirmação, que descreve corretamente a estrutura V x V da matriz. Além disso, aceita a quarta afirmação, confundindo o acesso indexado O(1) da matriz com a busca sequencial na lista.

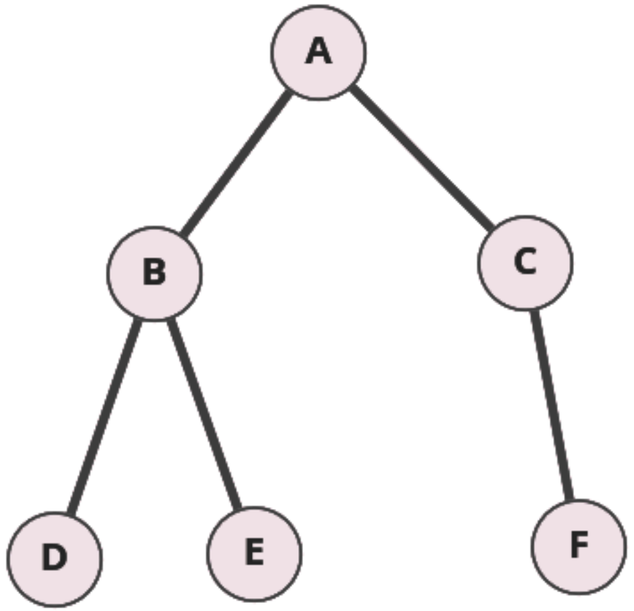
D) Acerta a avaliação das duas primeiras afirmações, mas inverte as duas últimas. Nega a economia de memória da lista em grafos esparsos e aceita a consulta direta em tempo constante na lista encadeada.

Capitulo 3

PERCURSO EM GRAFOS

DFS · BFS

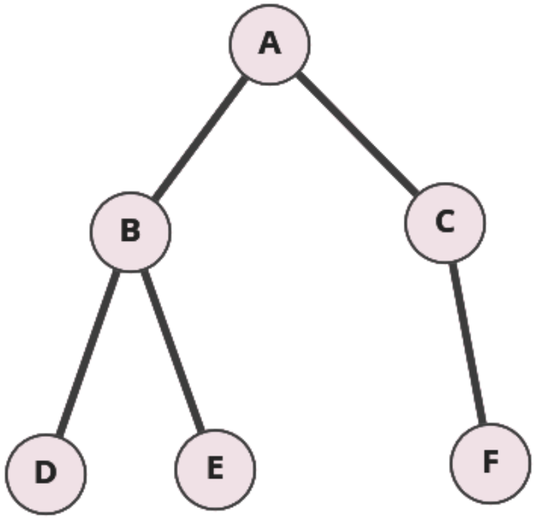
Como explorar um grafo?



A PERGUNTA:

COMO PERCORRER?

A partir de A, qual caminho devemos seguir?

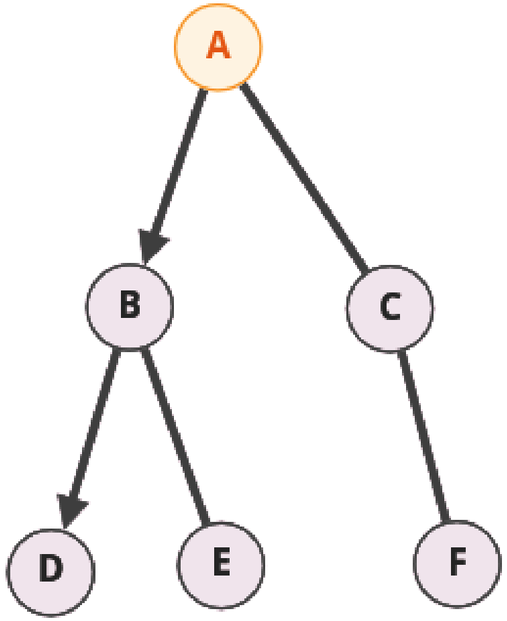


DUAS FORMAS DE EXPLORAR

DFS:

PROFUNDIDADE

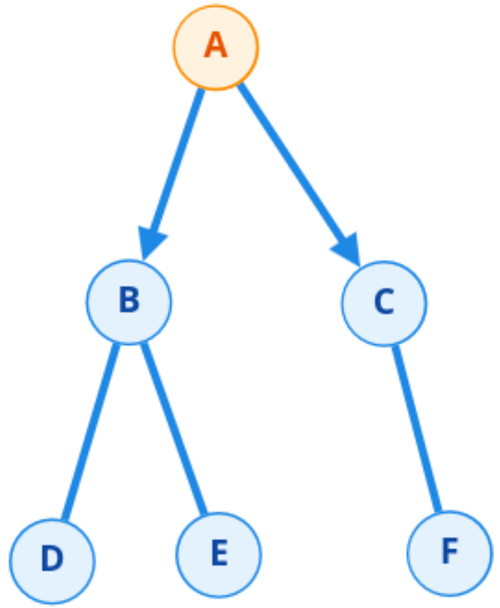
Vai o mais longe possível.



BFS:

LARGURA

Explora primeiro os mais próximos.

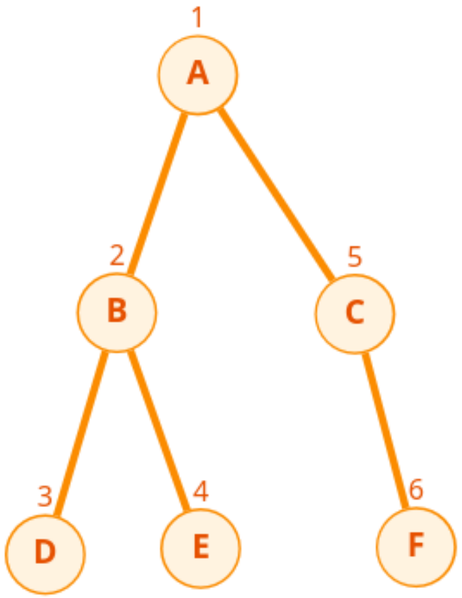


DFS

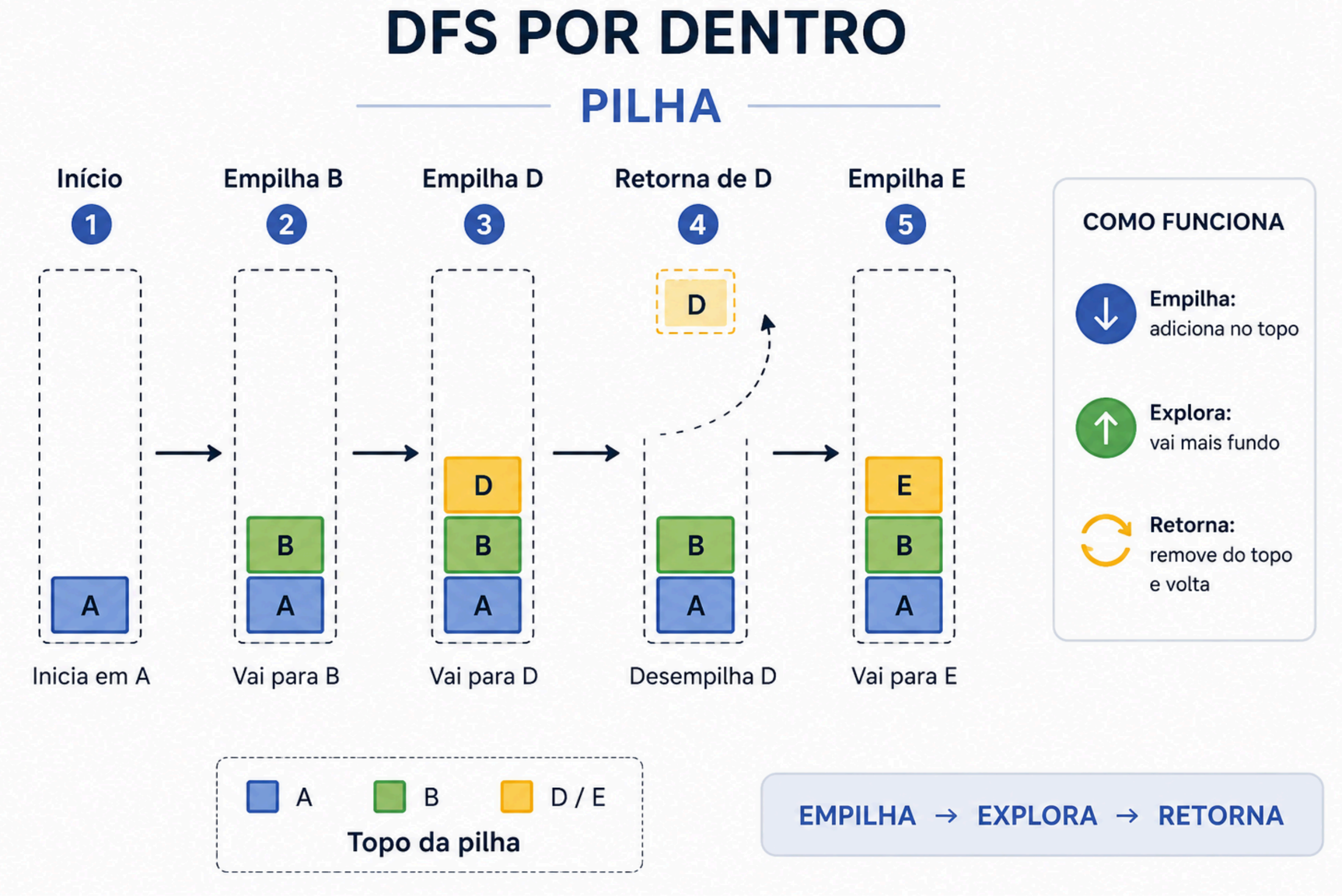
DEPTH-FIRST SEARCH

A → B → D → E → C → F

PROFUNDIDADE



Por que o DFS produz essa ordem?



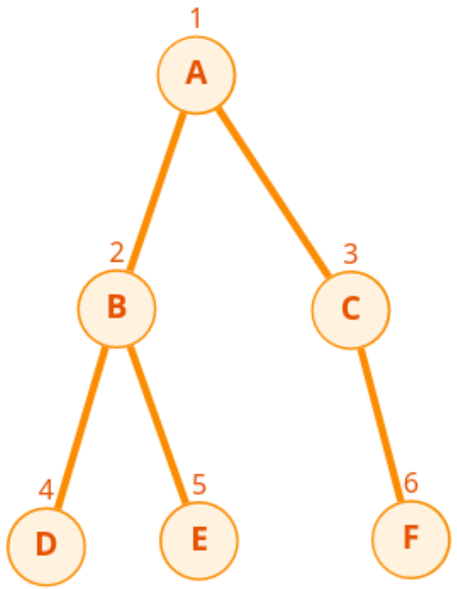
Visita um vértice e continua para um vizinho não visitado até não poder mais avançar.

BFS

BREADTH-FIRST SEARCH

A → B → C → D → E → F

LARGURA



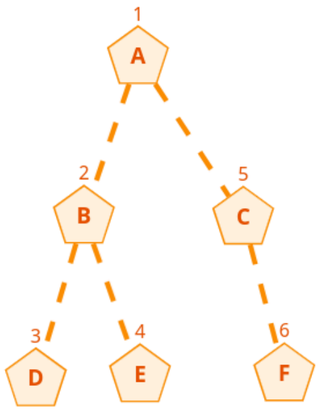
Por que o BFS produz essa ordem?



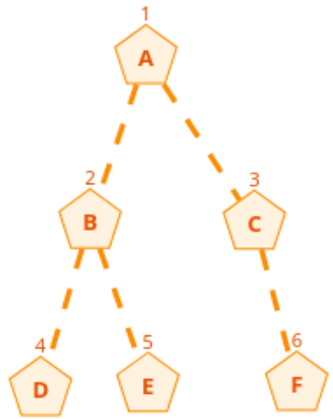
Visita primeiro todos os vizinhos do vértice atual antes de avançar para o próximo nível.

Comparação Técnica: DFS vs BFS

Resumo estrutural para resolução de questões no ENADE.



	DFS	BFS
Estratégia	Profundidade	Largura
Estrutura	Pilha / Recursão	Fila
Exploração	Um caminho por vez	Por níveis
Menor caminho não ponderado	✗	✓
Conectividade	✓	✓
Detecção de ciclos	✓	✓
Ordenação topológica	✓	✗

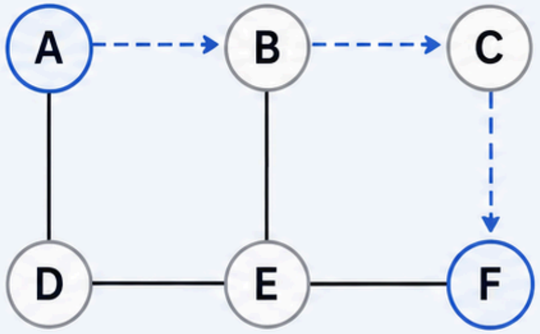


DFS aprofunda. BFS amplia.

PARA QUE SERVEM?

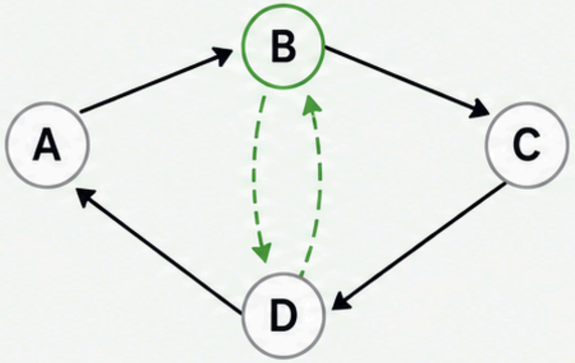
CONECTIVIDADE

Existe caminho entre dois vértices?



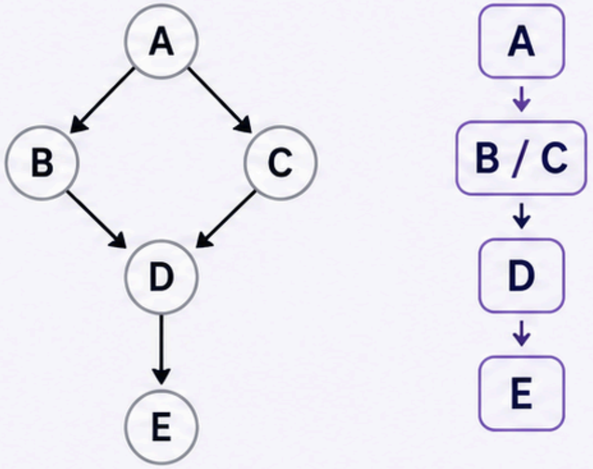
CICLOS

O caminho retorna a um vértice?



ORDENAÇÃO TOPOLÓGICA

Qual ordem respeita as dependências?



Conectividade e Detecção de Ciclos

Aplicações fundamentais dos percursos em grafos direcionados e não direcionados.

CONECTIVIDADE

Componentes Conexos

Determinar se existe caminho entre dois vértices e isolar subgrafos totalmente conectados.

Implementação: Disparo de DFS/BFS múltiplos a partir de nós não visitados.

DETECÇÃO DE CICLOS

Arestas de Retorno

Identificar laços onde um vértice aponta para um ancestral já presente na pilha de recursão atual.

Essencial para: Validação de consistência e prevenção de deadlocks.

ORDENAR DEPENDÊNCIAS

Organização linear de vértices em um grafo direcionado acíclico (DAG) respeitando dependências.

DAG — Grafo Direcionado Acíclico

PRINCÍPIO DE DEPENDÊNCIA

Precedência de Tarefas

Se existe uma aresta direcionada de U para V, U deve aparecer antes de V na sequência linear resultante.

Restrição: Só é possível em grafos sem ciclos (DAG).

RESOLUÇÃO COM DFS

Pós-Ordem Invertida

Adiciona os vértices a uma lista à medida que a recursão DFS é concluída (pós-ordem), invertendo o resultado ao final.

Aplicações: Compiladores, gerenciamento de pacotes e árvore de build.

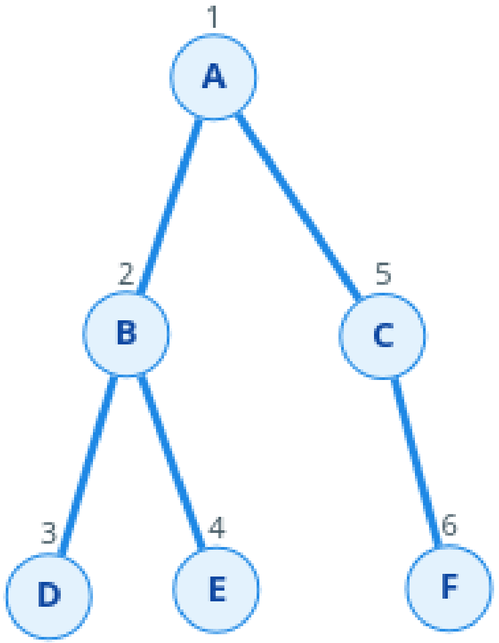
1 → A
2 → B
3 → D
4 → E
5 → C
6 → F

AGORA, NA PRÁTICA.

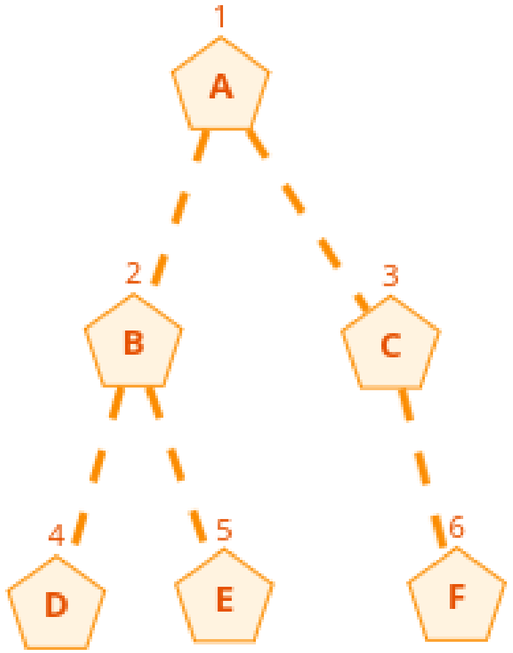
Mesmo grafo. Duas estratégias.

1 → A
2 → B
3 → C
4 → D
5 → E
6 → F

DFS



BFS



QUESTÃO 3 - ENADE

SITUAÇÃO-PROBLEMA

Uma empresa de transporte deseja analisar uma rede de estações representada por um grafo não ponderado, no qual cada vértice representa uma estação e cada aresta representa uma conexão direta entre duas estações.
Para determinar a menor quantidade de conexões necessárias para chegar de uma estação de origem S até uma estação de destino T, um analista decide utilizar o algoritmo Breadth-First Search (BFS).

COMANDO

Considerando o funcionamento da BFS em grafos não ponderados, analise as asserções a seguir.
I. A BFS pode ser utilizada para determinar um caminho entre S e T que contenha o menor número possível de arestas.

PORQUE

II. Durante sua execução, a BFS visita os vértices em níveis de distância crescente a partir da origem, utilizando uma fila para garantir que os vértices descobertos em uma determinada distância sejam processados antes dos vértices de distâncias maiores.
Assinale a alternativa correta.

ALTERNATIVAS

A) A asserção I é verdadeira, mas a II é falsa, pois a BFS utiliza uma pilha para determinar a ordem de visitação.
B) As asserções I e II são verdadeiras, mas a II não justifica corretamente a I.
C) As asserções I e II são verdadeiras, e a II justifica corretamente a I.
D) A asserção I é falsa, mas a II é verdadeira, pois a BFS não garante o menor número de arestas entre dois vértices.

QUESTÃO 3 - GABARITO

A) A asserção I é verdadeira, mas a II é falsa. A BFS não utiliza uma pilha para controlar a ordem de processamento. Ela utiliza uma fila FIFO. A utilização de uma pilha é característica da DFS (Depth-First Search).

B) As duas asserções são verdadeiras, porém a II justifica diretamente a I. A exploração em camadas com o uso da fila FIFO é o que garante o menor número de arestas.

C) As duas asserções são verdadeiras, e a II explica diretamente a I. A BFS explora os vértices em camadas de distância crescente a partir da origem e utiliza uma fila FIFO. Assim, os vértices mais próximos são processados antes dos mais distantes, garantindo que o caminho encontrado tenha o menor número possível de arestas em um grafo não ponderado.

D) A asserção II é verdadeira, mas a I também é verdadeira. Em um grafo não ponderado, a BFS garante um caminho com o menor número de arestas entre a origem e os demais vértices alcançáveis. Portanto, é incorreto afirmar que a BFS não possui essa garantia.

Capitulo 4

Algoritmos de Otimização

A PERGUNTA:

COMO TOMAR A MELHOR DECISÃO PARA OTIMIZAR RECURSOS EM UMA REDE COMPLEXA?

Menor Caminho:

- Dijkstra
- Bellman-Ford

Árvore Geradora Mínima:

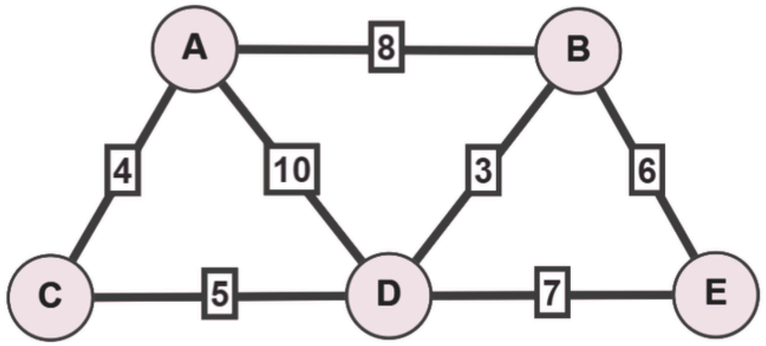
- Algoritmo de Prim

O Algoritmo de Dijkstra

A PERGUNTA:

COMO ENCONTRAR A ROTA MAIS RÁPIDA OU DE MENOR CUSTO PARTINDO DE UMA ORIGEM?

- Resolve o problema de menores caminhos a partir de uma única origem.
- Utiliza uma estratégia gulosa baseada na menor distância provisória.
- Restrição: Funciona apenas com pesos não negativos.

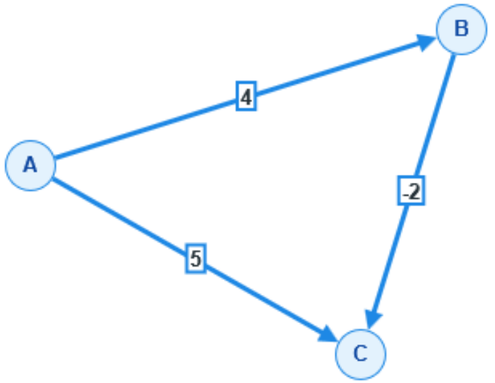


	A	B	C	D	E
A	0	8	4	10	∞
B	8	0	∞	3	6
C	4	∞	0	5	∞
D	10	3	5	0	7
E	∞	6	∞	7	0

O Algoritmo de Bellman-Ford

A PERGUNTA:

E SE A REDE POSSUIR CRÉDITOS OU ROTAS COM PESOS NEGATIVOS?



	A	B	C
A	0	4	5
B	∞	0	-2
C	∞	∞	0

- Resolve o problema de menores caminhos a partir de uma única origem.
- Relaxa repetidamente todas as arestas.
- Realiza até |V|-1 passadas para consolidar as menores distâncias.
- Admite arestas de peso negativo.
- Detecta ciclos de peso negativo alcançáveis a partir da origem.

Comparação Técnica: Dijkstra vs Bellman-Ford

CARACTERÍSTICA	DIJKSTRA	BELLMAN-FORD
Estratégia principal	Estratégia gulosa (escolhe a menor distância provisória)	Relaxamento repetido de todas as arestas
Passagens principais	Uma vez por vértice	Até |V| -1 passadas
Pesos negativos	Não suporta (falha ou gera resultados incorretos)	Suporta e admite arestas com peso negativo
Ciclos negativos	Não é capaz de detectar	Detecta ciclos de peso negativo alcançáveis a partir da origem
Objetivo	Menores caminhos a partir de uma única origem	Menores caminhos a partir de uma única origem

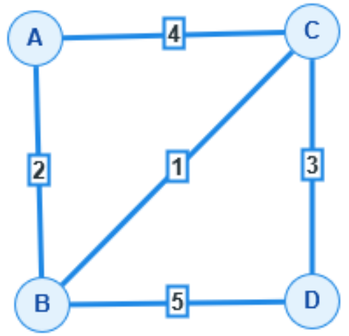
ALGORITMO DE PRIM

A PERGUNTA:

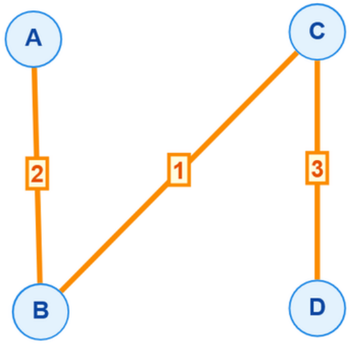
COMO CONECTAR TODOS OS VÉRTICES GASTANDO O MÍNIMO POSSÍVEL?

- Inicia em um vértice.
- A cada passo, seleciona a aresta de menor peso que conecta um vértice já visitado a um vértice ainda não visitado.
- O processo continua até que todos os vértices estejam conectados.
- O resultado é uma Árvore Geradora Mínima (MST).

Antes:



Depois:



Prim na Prática: Rastreamento

Passo	Vértices Conectados	Arestas Avaliadas (Fronteira)	Aresta Escolhida	Custo Parcial
1	{A}	A-B (2), A-C (4)	**A-B (2)**	2
2	{A, B}	A-C (4), B-C (1), B-D (5)	**B-C (1)**	3
3	{A, B, C}	A-C (descarta, gera ciclo), B-D (5), C-D (3)	**C-D (3)**	6
Fim	{A, B, C, D}	Todos os vértices interligados	**N/A**	**Custo Total: 6**

QUESTÃO 4 - ENADE

SITUAÇÃO-PROBLEMA

Uma equipe de engenharia de software está arquitetando a infraestrutura de rede para um ecossistema de microsserviços em nuvem, semelhante a ambientes AWS com contêineres Docker. Durante o planejamento, os engenheiros de backend identificaram três desafios distintos de otimização de rede que precisam ser resolvidos de forma automatizada:

- **Desafio 1: Encontrar a rota de menor latência (tempo de resposta) entre o API Gateway (nó de origem) e todos os outros microsserviços da rede. Sabe-se que todos os links possuem latência estritamente positiva.**

- **Desafio 2: Calcular o custo de roteamento de sincronização de um banco de dados relacional distribuído. Devido a acordos de tráfego com provedores, alguns links geram "créditos" de banda, operando efetivamente como arestas de peso negativo. É necessário encontrar o menor custo a partir do nó primário e garantir que não existam ciclos de retroalimentação infinita (ciclos negativos).**

- **Desafio 3: Interligar fisicamente todos os servidores de um novo data center com cabos de fibra óptica. O objetivo não é a velocidade entre dois pontos específicos, mas sim garantir que todos os servidores estejam conectados na mesma rede gastando o mínimo possível em metros de cabo.**

COMANDO

Considerando os algoritmos clássicos de otimização em Teoria dos Grafos, assinale a alternativa que indica a escolha tecnicamente correta para resolver os Desafios 1, 2 e 3, respectivamente.

ALTERNATIVAS

A) 1 - Bellman-Ford; 2 - Dijkstra; 3 - Prim.
B) 1 - Dijkstra; 2 - Bellman-Ford; 3 - Prim.
C) 1 - Dijkstra; 2 - Prim; 3 - Bellman-Ford.
D) 1 - Prim; 2 - Bellman-Ford; 3 - Dijkstra.

QUESTÃO 4 - GABARITO

A) A alternativa A está incorreta porque, embora o Bellman-Ford possa resolver o Desafio 1, o Dijkstra é mais eficiente quando todos os pesos são positivos. No Desafio 2, o Dijkstra não é adequado porque existem arestas de peso negativo. Nesse caso, o algoritmo correto é o Bellman-Ford.

B) A alternativa B está correta porque o Dijkstra é adequado para encontrar os menores caminhos a partir de uma única origem quando todos os pesos são positivos. Para o segundo desafio, o Bellman-Ford é necessário, pois suporta arestas de peso negativo e detecta ciclos negativos alcançáveis a partir da origem. Já o terceiro desafio corresponde ao problema de Árvore Geradora Mínima, em que o objetivo é conectar todos os vértices com o menor custo total, sendo o Prim uma solução adequada.

C) A alternativa C está incorreta porque, apesar de utilizar corretamente o Dijkstra no Desafio 1, utiliza o Prim no Desafio 2, que não resolve problemas de menores caminhos com pesos negativos. Além disso, o Bellman-Ford não é destinado à construção de uma Árvore Geradora Mínima, que é o objetivo do Desafio 3.

D) A alternativa D está incorreta porque utiliza o Prim no Desafio 1, que não tem como objetivo encontrar menores caminhos a partir de uma origem. Embora o Bellman-Ford esteja correto no Desafio 2, o Dijkstra está incorreto no Desafio 3, pois esse problema busca conectar todos os vértices com o menor custo total, caracterizando uma Árvore Geradora Mínima.

REFERÊNCIAS

BONDY, J. A.; MURTY, U. S. R. Graph Theory. New York: Springer, 2008.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4. ed. Cambridge, MA: MIT Press, 2022.

DIESTEL, Reinhard. Graph Theory. 5. ed. Berlin: Springer, 2017.

ROSEN, Kenneth H. Discrete Mathematics and Its Applications. 8. ed. New York: McGraw-Hill, 2019.

WEST, Douglas B. Introduction to Graph Theory. 2. ed. Upper Saddle River: Prentice Hall, 2001.

OBRIGADO!!